

Famto Platform

Production-Grade Microservice Architecture Plan

Version 2.0 | June 02, 2026 | 2 Developers | 52-Week Roadmap

CURRENT MONOLITH	TARGET ARCHITECTURE
22,500 lines / single process	8 microservices / independent deploy
No caching, no queues	Redis + RabbitMQ + Kafka
4,974-line socket.js	Dedicated Notification Service
console.log debugging	Pino + ELK Stack observability
Single Dockerfile	Kubernetes + Helm + HPA autoscaling
No circuit breakers	Resilience patterns + health probes
Shared MongoDB connection	Database-per-service isolation
No API contracts	OpenAPI specs + contract testing

Table of Contents

- 1. Executive Summary
- 2. System Architecture Overview
- 3. Service Catalog & Domain Boundaries
- 4. Infrastructure Stack
- 5. Inter-Service Communication Patterns
- 6. Data Architecture & Migration
- 7. Observability & Monitoring
- 8. Security Architecture
- 9. CI/CD Pipeline Design
- 10. Kubernetes Deployment Topology
- 11. Resilience & Fault Tolerance
- 12. Phase 0 — Foundation Hardening (Weeks 1-6)
- 13. Phase 1 — Notification Service (Weeks 7-12)
- 14. Phase 2 — Auth Service + API Gateway (Weeks 13-16)
- 15. Phase 3 — Pricing & Billing Service (Weeks 17-22)
- 16. Phase 4 — Merchant Service (Weeks 23-28)
- 17. Phase 5 — Customer + Agent Services (Weeks 29-36)
- 18. Phase 6 — Order Service + Kafka (Weeks 37-46)
- 19. Phase 7 — Kubernetes + Decommission (Weeks 47-52)
- 20. Developer Allocation Matrix
- 21. Risk Register
- 22. Acceptance Criteria & Definition of Done

1. Executive Summary

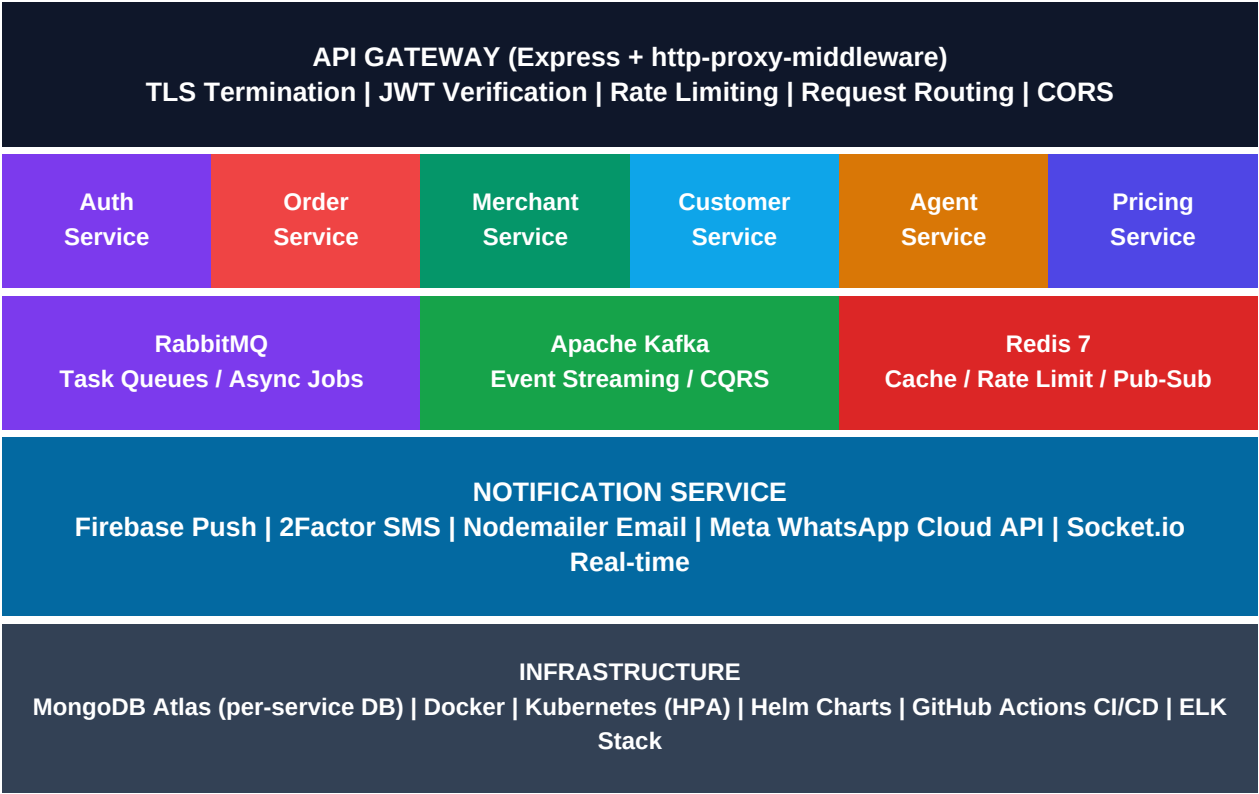
Famto is a food-delivery and logistics platform serving customers, merchants, and delivery agents across multiple Indian cities. The current backend is a monolithic Node.js/Express application with 22,500 lines of code, 79 Mongoose models, and a 4,974-line real-time Socket.io handler. It lacks structured logging, caching, message queues, database transactions, and automated tests.

This document defines a **52-week migration roadmap** to decompose the monolith into **8 independently deployable microservices** running on Kubernetes, connected via Kafka (event streaming), RabbitMQ (task queues), and Redis (caching/pub-sub). The migration uses the **Strangler Fig** pattern — extracting one service at a time while production traffic continues uninterrupted.

Key Outcomes

- Independent deployment — ship each service without coordinated releases
- Horizontal scaling — auto-scale Order and Notification services under load
- Fault isolation — a crash in Notification does not take down Order processing
- Technology flexibility — each service can adopt best-fit libraries independently
- Team autonomy — 2 developers can own and evolve services without merge conflicts

2. System Architecture Overview



3. Service Catalog & Domain Boundaries

Service	Bounded Context	Models Owned	External Deps	Scale Profile
API Gateway	Routing, auth verify, rate limit	None (stateless)	Redis	2 replicas (always-on)
Auth	Identity, JWT, OTP, password reset	Admin, Manager, ManagerRoles, Token	2Factor.in SMS, Redis	2 replicas
Order	Order lifecycle, cart, payment, scheduling, tasks	Order, TemporaryOrder, ScheduledOrder, Task, PickAndCustomCart, DatabaseCounter	Razorpay, all other services	3-8 replicas (HPA)
Merchant	Merchant profiles, products, categories, discounts, banners, payouts	Merchant, Category, Product, BusinessCategory, Discounts, Banners, MerchantPayout	Firebase Storage	2-4 replicas
Customer	Customer profiles, addresses, wallet, loyalty, referrals, promo codes	Customer, CustomerCart, Wallet, LoyaltyPoint, Referral, PromoCode	Razorpay (wallet)	2-4 replicas
Agent	Agent profiles, location, task mgmt, earnings, work history	Agent, AgentTransaction, AgentWorkHistory, AutoAllocation	MapMyIndia/Mappls	2-6 replicas (HPA)
Pricing & Billing	Dynamic pricing, surge, tax, commission, subscriptions, revenue	CustomerPricing, MerchantPricing, AgentPricing, *Surge, Tax, Commission, Subscription	None	2 replicas
Notification	Push, SMS, email, WhatsApp, Socket.io real-time, notification logs	NotificationSettings, FCMTOKEN, all NotificationLogs, 8 Whatsapp* models	Firebase FCM, 2Factor, Meta WhatsApp API	3-6 replicas (HPA)

4. Infrastructure Stack

Layer	Technology	Purpose	Configuration
Runtime	Node.js 20 LTS (Alpine)	Service runtime	--max-old-space-size=512
Framework	Express 4.x	HTTP framework	Helmet, CORS, compression
Database	MongoDB Atlas 7.x	Primary data store	Per-service DB, replica set, WiredTiger
Cache	Redis 7.x	Caching, rate limiting, pub/sub	Cluster mode, maxmemory-policy: allkeys-lru
Message Queue	RabbitMQ 3.13	Task queues, async jobs	Durable queues, dead-letter exchange, prefetch=10
Event Stream	Apache Kafka 3.7	Event sourcing, CQRS	3 brokers, replication-factor=3, retention=7d
Container	Docker 25.x	Containerization	Multi-stage builds, non-root user, distroless
Orchestration	Kubernetes 1.29	Container orchestration	HPA, PDB, resource limits, liveness/readiness
Package Mgr	Helm 3.x	K8s deployment management	Per-service chart + shared library chart
CI/CD	GitHub Actions	Build, test, deploy pipeline	Per-service workflows, semantic versioning
Logging	Pino + ELK Stack	Structured logging + search	JSON format, request-id correlation, 30d retention
Metrics	Prometheus + Grafana	Metrics collection + dashboards	RED metrics (Rate, Errors, Duration)
Tracing	OpenTelemetry + Jaeger	Distributed tracing	Auto-instrumentation, 1% sampling in prod
Secrets	K8s Secrets + sealed-secrets	Secret management	Encrypted at rest, RBAC-scoped
DNS/Ingress	NGINX Ingress Controller	L7 load balancing, TLS	cert-manager for Let's Encrypt auto-renewal

5. Inter-Service Communication Patterns

Three communication patterns are used based on the coupling and latency requirements:

5.1 Synchronous — HTTP/REST (Request-Response)

Used when the caller **needs an immediate response** to continue processing.

Caller	Callee	Endpoint	When
Order Service	Customer Service	GET /internal/customers/:id/validate	Order creation — verify customer exists and has wallet balance
Order Service	Merchant Service	GET /internal/merchants/:id/products	Order creation — validate products and check stock
Order Service	Pricing Service	GET /internal/pricing/calculate	Order creation — get delivery charge, surge, tax
Order Service	Agent Service	GET /internal/agents/available?geofenceId=X	Auto-allocation — find nearby available agents
API Gateway	Auth Service	POST /internal/auth/verify-token	Only for token refresh; JWT signature verified locally

Resilience: All synchronous calls use circuit breakers (opossum), 3s timeout, 2 retries with exponential backoff.

5.2 Asynchronous — RabbitMQ (Task Queues)

Used for **fire-and-forget tasks** where the caller does not need the result.

Queue	Producer	Consumer	Purpose
notification.push	Order, Merchant, Agent	Notification Service	Firebase push notifications
notification.sms	Auth Service	Notification Service	OTP and verification SMS
notification.email	Order Service	Notification Service	Order confirmation, invoice emails
notification.whatsapp	Order, Customer	Notification Service	WhatsApp template messages
notification.socket	All services	Notification Service	Real-time Socket.io events to dashboard/apps

Queue	Producer	Consumer	Purpose
campaign.send	Notification Service (API)	Notification Service (worker)	Bulk WhatsApp campaign processing

Config: Durable queues, manual ack, prefetch=10, dead-letter exchange (DLX) for failed messages, TTL=24h on DLX, alerting when DLQ depth > 100.

5.3 Event-Driven — Apache Kafka (Domain Events)

Used for **domain events** that multiple services may need to react to.

Topic	Producer	Consumers	Schema
order.events	Order Service	Notification, Pricing, Analytics	{ orderId, status, merchantId, customerId, agentId, timestamp }
merchant.events	Merchant Service	Order, Pricing	{ merchantId, event, data, timestamp }
agent.events	Agent Service	Order, Notification	{ agentId, event, location, timestamp }
payment.events	Order Service	Customer, Merchant, Notification	{ orderId, paymentId, status, amount, timestamp }

Config: 3 partitions per topic, replication-factor=3, retention=7 days, consumer groups per service, exactly-once semantics via idempotent producers + transactional consumers.

6. Data Architecture & Migration

6.1 Database-Per-Service Strategy

Each microservice owns its database. No service accesses another service's database directly. Cross-service data is fetched via APIs or consumed via Kafka events.

Service	Database Name	Key Collections	Indexes Required
Auth	famto_auth	admins, managers, managerroles, tokens	email (unique), phoneNumber (unique)
Order	famto_orders	orders, temporaryorders, scheduledorders, tasks	(merchantId,status), (customerId,createdAt), (agentId,status), (paymentStatus), (status,createdAt)
Merchant	famto_merchants	merchants, categories, products, discounts, banners	2dsphere on geoLocation, (status,isApproved), (categoryId), name text index
Customer	famto_customers	customers, carts, wallets, loyaltypoints, referrals	(phoneNumber unique), (email sparse), (referralCode unique)
Agent	famto_agents	agents, transactions, workhistories, autoallocations	2dsphere on location, (status,isApproved), (geofenceId,isAvailable)
Pricing	famto_pricing	pricings, surges, taxes, commissions, subscriptions	(geofenceId,vehicleType), (businessCategoryId)
Notification	famto_notifications	settings, fcmtokens, logs, whatsapp_*	(userId,createdAt), (walId), (metaMessageId sparse unique)

6.2 Migration Phases

Phase	Strategy	Downtime	Rollback Plan
Phase 0-2 (Weeks 1-16)	All services share single MongoDB Atlas database (famto_db). No migration needed.	Zero	N/A — same DB
Phase 3-5 (Weeks 17-36)	Each extracted service gets its own database on same Atlas cluster. One-time mongodump/mongorestore per collection. Monolith stops writing to migrated collections.	< 5 min per service (read-only window)	Revert gateway routes to monolith; monolith still has code

Phase	Strategy	Downtime	Rollback Plan
Phase 6-7 (Weeks 37-52)	Full database isolation. Cross-service reads via HTTP APIs. Eventual consistency via Kafka events for denormalized views.	Zero (dual-write during transition)	Kafka consumer replay from offset

6.3 Cross-Service Data Access Patterns

- Order Service needs customer name/phone for receipts -> calls Customer Service API (cached in Redis for 5 min)
- Order Service needs merchant name/address for delivery -> calls Merchant Service API (cached in Redis for 10 min)
- Notification Service needs user FCM tokens -> owns them directly (no cross-service call)
- Dashboard needs aggregated stats -> dedicated read models built from Kafka events (CQRS)

7. Observability & Monitoring

7.1 Structured Logging (Pino + ELK)

Every service uses Pino for structured JSON logging. All logs include: timestamp, level, service name, request-id (X-Request-ID header propagated across services), user ID, and trace ID.

```
{ "level": 30, "time": 1717200000, "service": "order-service", "reqId": "abc-123",
  "msg": "Order created", "orderId": "0260601001", "userId": "66a34f04...", "traceId":
  "def-456" }
```

7.2 Metrics (Prometheus + Grafana)

RED metrics for every service:

Metric	Type	Labels	Alert Threshold
http_request_duration_seconds	Histogram	service, method, route, status_code	p99 > 2s for 5 min
http_requests_total	Counter	service, method, route, status_code	5xx rate > 1% for 5 min
rabbitmq_queue_depth	Gauge	queue_name	Depth > 1000 for 10 min
kafka_consumer_lag	Gauge	topic, consumer_group	Lag > 10000 for 5 min
mongodb_connection_pool_size	Gauge	service	Available < 5 for 5 min
redis_hit_rate	Gauge	service	Hit rate < 80% for 30 min
nodejs_heap_used_bytes	Gauge	service	Used > 400MB for 10 min

7.3 Distributed Tracing (OpenTelemetry + Jaeger)

- Auto-instrumentation via @opentelemetry/auto-instrumentations-node
- Trace context propagated via W3C traceparent header across HTTP and Kafka
- 1% sampling in production, 100% in staging
- Jaeger UI for visualizing cross-service request flows

7.4 Health Checks

Endpoint	Purpose	K8s Probe	Check Interval
GET /health/live	Process is running	livenessProbe	Every 10s, failureThreshold=3

Endpoint	Purpose	K8s Probe	Check Interval
GET /health/ready	Service can handle traffic (DB + Redis connected)	readinessProbe	Every 5s, failureThreshold=2
GET /health/startup	Initial startup complete	startupProbe	Every 5s, failureThreshold=30

8. Security Architecture

8.1 Authentication & Authorization

Layer	Mechanism	Implementation
External -> Gateway	JWT Bearer token	Gateway validates JWT signature (shared secret). Rejects expired/blacklisted tokens (Redis check).
Gateway -> Services	X-User-ID + X-User-Role headers	Gateway strips client auth header, injects verified user identity headers. Services trust gateway.
Service -> Service	Internal API key + network policy	K8s NetworkPolicy restricts pod-to-pod traffic. Internal endpoints require X-Internal-Key header.
Customer/Agent auth	OTP via 2Factor.in	Auth Service handles OTP send/verify. Returns JWT on success.
Admin/Merchant auth	Email + bcrypt password	Auth Service handles login. Returns JWT with role claim.

8.2 Security Controls

- Rate limiting: 100 req/min per IP at gateway (Redis-backed, express-rate-limit)
- Input validation: Zod schemas on every endpoint (replaces express-validator)
- Helmet.js: Security headers (X-Frame-Options, CSP, HSTS, X-Content-Type-Options)
- CORS: Whitelist production domains only (dashboard.famto.in, famto.in, order.famto.in)
- Secrets: Kubernetes Secrets (encrypted at rest) — never in .env files in production
- Dependency audit: npm audit in CI pipeline, fail build on critical vulnerabilities
- MongoDB: Atlas IP whitelist + SCRAM authentication + TLS in transit
- Container security: Non-root user in Dockerfile, read-only filesystem, no privileged mode

9. CI/CD Pipeline Design

Each microservice has its own GitHub Actions workflow. Builds trigger only when files in that service's directory change.

Pipeline Stages

Stage	Actions	Failure Behavior
1. Lint	ESLint + Prettier check	Block merge
2. Unit Test	Jest --coverage (min 60%)	Block merge
3. Integration Test	Supertest against service + test DB	Block merge
4. Security Scan	npm audit --audit-level=critical	Block merge on critical
5. Build	Docker multi-stage build (builder -> distroless)	Block deploy
6. Push	Push to container registry (GHCR or DigitalOcean)	Block deploy
7. Deploy Staging	Helm upgrade --install to staging namespace	Block prod deploy
8. Smoke Test	Health check + 3 critical API calls against staging	Block prod deploy
9. Deploy Prod	Helm upgrade --install to production namespace (manual approval)	Auto-rollback on failed readiness probe

Branching Strategy: trunk-based development. Feature branches merge to main via PR. Main auto-deploys to staging. Production deploy requires manual approval in GitHub Actions.

10. Kubernetes Deployment Topology

Service	Replicas (min/max)	CPU Request/Limit	Memory Request/Limit	HPA Metric
API Gateway	2 / 4	100m / 250m	128Mi / 256Mi	CPU > 70%
Auth Service	2 / 3	100m / 250m	128Mi / 256Mi	CPU > 70%
Order Service	3 / 8	250m / 500m	256Mi / 512Mi	CPU > 60%, RPS > 100
Merchant Service	2 / 4	100m / 300m	128Mi / 384Mi	CPU > 70%
Customer Service	2 / 4	100m / 300m	128Mi / 384Mi	CPU > 70%
Agent Service	2 / 6	150m / 400m	192Mi / 384Mi	CPU > 65%
Pricing Service	2 / 3	100m / 250m	128Mi / 256Mi	CPU > 70%
Notification Service	3 / 6	200m / 400m	256Mi / 512Mi	RabbitMQ queue depth > 500

Pod Disruption Budgets: minAvailable=1 for all services. Order and Notification have minAvailable=2 to ensure availability during rolling updates.

Node Affinity: Stateful infrastructure (MongoDB, Redis, Kafka, RabbitMQ) runs on dedicated node pools with SSD storage. Application pods run on general-purpose nodes.

11. Resilience & Fault Tolerance

Pattern	Implementation	Configuration
Circuit Breaker	opossum library on all inter-service HTTP calls	timeout=3s, errorThresholdPercentage=50, resetTimeout=30s, rollingCountTimeout=10s
Retry with Backoff	axios-retry on idempotent GET requests	retries=2, retryDelay=exponentialDelay, retryCondition=5xx or network error
Timeout	Express request timeout middleware	API routes: 10s, Internal routes: 5s, Webhook routes: 30s
Bulkhead	Separate connection pools per downstream service	maxSockets=10 per service, prevents one slow service from exhausting connections
Dead Letter Queue	RabbitMQ DLX for failed message processing	Max retries=3, then route to DLQ. Alert on DLQ depth > 100.
Graceful Shutdown	SIGTERM handler in each service	Stop accepting requests -> drain connections (30s) -> close DB/Redis -> exit
Idempotency	Idempotency key on Order creation + payment	Redis-based idempotency key with 24h TTL. Prevents duplicate orders on network retry.
Rate Limiting	express-rate-limit + rate-limit-redis	100 req/min per IP at gateway, 1000 req/min per authenticated user

PHASE 0 Foundation Hardening (Weeks 1-6)

Week 1-2: Logging + Error Handling + Graceful Shutdown

- Dev A: Install Pino + pino-http. Replace all 268+ console.log calls. Add request-id correlation.
- Dev B: Enhance globalErrorHandler.js (4xx vs 5xx). Add unhandledRejection/uncaughtException handlers. Implement SIGTERM graceful shutdown.

Week 3-4: Redis + Docker Compose

- Dev A: Add Redis. Replace in-memory distanceCache + node-cache with Redis. Add rate limiting (express-rate-limit + rate-limit-redis).
- Dev B: Create docker-compose.yml (app, MongoDB, Redis). Add /health endpoint. Move Firebase SDK init to shared config/firebaseAdmin.js.

Week 5-6: Database Optimization + Service Layer

- Dev A: Audit 79 models for missing indexes. Add .lean() to all read queries. Fix N+1 patterns in socket.js.
- Dev B: Create services/ + repositories/ directories. Refactor 2 controllers as template implementations. Write CONTRIBUTING.md.

Deliverable: Structured logging, Redis, Docker Compose, health checks, DB indexes, graceful shutdown.

PHASE 1 Extract Notification Service (Weeks 7-12)

Week 7-8: Service Scaffold + RabbitMQ

- Dev A: Create notification-service project. Add RabbitMQ to docker-compose. Define 5 queues.
- Dev B: Extract `sendPushNotificationToUser()`, `createNotificationLog()`, `sendSocketData()` from `socket.js`. Move Firebase SDK.

Week 9-10: Event-Driven Integration

- Dev A: Replace 20+ direct notification calls in monolith with RabbitMQ publishers.
- Dev B: Migrate Socket.io server to Notification Service with Redis adapter. Monolith -> RabbitMQ -> Notification -> Socket.io.

Week 11-12: WhatsApp + Testing

- Dev A: Move 9 WhatsApp controllers + 8 models into Notification Service.
- Dev B: Integration test all 22 socket events. Load test RabbitMQ throughput.

Deliverable: Notification Service running independently. RabbitMQ connecting monolith to notifications.

PHASE 2 Extract Auth Service + API Gateway (Weeks 13-16)

Week 13-14: Auth Service

- Dev A: Create Auth Service — login, OTP, register, refresh-token, forgot-password endpoints.
- Dev B: Create shared JWT validation middleware (local signature verification). Redis token blacklist.

Week 15-16: API Gateway

- Dev A: Set up Express API Gateway with http-proxy-middleware. Route auth + whatsapp to new services.
- Dev B: Remove auth code from monolith. Test all 5 actor login flows.

Deliverable: Auth Service + API Gateway running. All traffic flows through gateway.

PHASE 3 Extract Pricing & Billing (Weeks 17-22)

Week 17-20: Pricing + Subscription

- Dev A: Move 6 pricing + 6 surge controllers + Tax/Commission/Subscription models.
- Dev B: Expose GET /internal/pricing/calculate API. Move billing cron jobs.

Week 21-22: Revenue + Testing

- Dev A: Move homeController dashboard aggregations + revenue helpers.
- Dev B: Verify pricing calculations match monolith exactly (characterization tests).

Deliverable: Pricing Service running. Monolith calls Pricing API for all calculations.

PHASE 4 Extract Merchant Service (Weeks 23-28)

Week 23-26: Merchant Core + APIs

- Dev A: Move Merchant, Product, Category, BusinessCategory models + controllers.
- Dev B: Move discounts, banners, app customization. Expose internal APIs. Publish Kafka events.

Week 27-28: Payout + Testing

- Dev A: Move merchant payout logic + cron jobs.
- Dev B: E2E testing — product browsing, merchant status, order creation with Merchant Service data.

Deliverable: Merchant catalog served from Merchant Service.

PHASE 5 Extract Customer + Agent Services (Weeks 29-36)

Week 29-31: Customer Service

- Dev A: Move Customer, Cart, Wallet, LoyaltyPoint, Referral, PromoCode models.
- Dev B: Move customer controllers (excluding order-related). Expose validate + wallet APIs.

Week 32-34: Agent Service

- Dev A: Move Agent, AgentTransaction, AgentWorkHistory, AutoAllocation models.
- Dev B: Move agent controllers (excluding order-accept/reject). Expose available agents API.

Week 35-36: Integration Testing

- Both devs: Full integration testing across all 6 extracted services.

Deliverable: Customer + Agent Services running independently.

PHASE 6 Extract Order Service + Kafka (Weeks 37-46)

Week 37-38: Kafka Setup

- Dev A: Add Kafka to infrastructure. Define 4 event topics with schemas.
- Dev B: Refactor monolith order flow to service pattern. Map all cross-service dependencies.

Week 39-42: Order Service Core

- Dev A: Move Order, TemporaryOrder, ScheduledOrder, Task, DatabaseCounter models.
- Dev B: Replace direct model imports with inter-service HTTP calls + circuit breakers.

Week 43-44: Payment + Socket Events

- Dev A: Move Razorpay integration + webhook handler.
- Dev B: Move order socket events. Order -> Kafka -> Notification Service.

Week 45-46: Cron Migration + E2E

- Dev A: Move all order cron jobs from index.js.
- Dev B: Full lifecycle test — all 4 delivery modes, scheduled orders, payment failures.

Deliverable: Order Service is source of truth. Monolith is now just the gateway.

PHASE 7 Kubernetes + Decommission (Weeks 47-52)

Week 47-48: Monolith Cleanup

- Both: Remove all extracted code. Remaining admin ops become thin Admin Service or stay in gateway.

Week 49-50: Kubernetes

- Dev A: Create K8s manifests — Deployments, Services, ConfigMaps, Secrets, HPA, PDB. Helm charts.
- Dev B: CI/CD pipeline per service with GitHub Actions. Container registry. Staging environment.

Week 51-52: Production Deploy

- Both: Blue-green deployment to K8s cluster. Old monolith as fallback for 2 weeks. Monitor + decommission.

Deliverable: All 8 services on Kubernetes. Monolith decommissioned.

20. Developer Allocation Matrix

Week	Dev A	Dev B
1-2	Structured logging (Pino)	Graceful shutdown + error handling
3-4	Redis caching + rate limiting	Docker Compose + Firebase refactor + /health
5-6	DB indexes + .lean() + N+1 fixes	Service layer pattern + 2 controller refactors
7-8	Notification Service scaffold + RabbitMQ setup	Extract notification functions from socket.js
9-10	RabbitMQ producers in monolith (20 files)	Socket.io migration to Notification Service
11-12	WhatsApp module migration (9 controllers + 8 models)	Integration testing all 22 socket events
13-14	Auth Service (login, OTP, register, refresh)	JWT shared middleware + Redis token blacklist
15-16	API Gateway (Express + http-proxy-middleware)	Monolith auth removal + 5 actor login tests
17-20	Pricing Service core (12 controllers + models)	Internal pricing API + subscription migration
21-22	Revenue/analytics migration	Characterization tests for pricing
23-26	Merchant profiles + products + internal APIs	Discounts, banners + Kafka event publishing
27-28	Payout logic migration	E2E testing (product browse -> order)
29-31	Customer Service (models + controllers)	Wallet + loyalty + referral migration
32-34	Agent Service (models + controllers)	Location tracking + auto-allocation migration
35-36	Cross-service integration testing	Cross-service integration testing
37-38	Kafka setup + event schema design	Order flow refactor to service pattern
39-42	Order Service core (5 model groups)	Inter-service calls + circuit breakers

Wee k	Dev A	Dev B
43-4 4	Order socket events migration	Razorpay payment migration
45-4 6	Order cron job migration	E2E order lifecycle testing (4 modes)
47-4 8	Monolith cleanup	Admin Service remainder
49-5 0	K8s manifests + Helm charts + HPA	CI/CD pipeline + container registry
51-5 2	Production blue-green deployment	Monitoring + fallback + decommission

21. Risk Register

Risk	Likelihood	Impact	Mitigation
Socket.io migration breaks real-time features (live tracking, dashboard)	High	Critical	Feature flag to toggle old/new. Keep monolith Socket.io as fallback for 2 weeks. Test all 22 events individually.
Order creation race conditions during service split	High	Critical	Redis distributed locks (Redlock) for order creation. MongoDB transactions. Never two services writing to same order.
Team burnout from 52-week migration	High	High	Strict phase gates — no new phase until previous is stable in production for 1 week. Allow bug-fix-only weeks.
Data inconsistency between service databases	Medium	Critical	Start with shared DB (Phase 0-2). Separate DBs only after boundary is proven. Kafka dead-letter queues with alerting.
Inter-service HTTP latency degrades order creation	Medium	Medium	Redis caching for hot data (pricing: 5min TTL, merchant details: 10min TTL). Connection pooling. Batch API calls.
Kafka/RabbitMQ operational complexity overwhelms team	Medium	Medium	Start with RabbitMQ only (Phase 1). Add Kafka only in Phase 6. Use managed services if available.
DatabaseCounter (custom ID generation) conflicts across services	Low	Critical	Keep DatabaseCounter in Order Service only (atomic findOneAndUpdate with \$inc). No other service generates order IDs.
Frontend team blocks on Socket.io URL change	Low	Medium	Only one frontend change required (Phase 1). Coordinate 1 week ahead. Env var config, not hard-coded.

22. Acceptance Criteria & Definition of Done

Per-Phase Definition of Done

- All integration tests pass against the new service
- API responses match monolith exactly (characterization test comparison)
- Error rate in production < 0.1% for 7 consecutive days
- No increase in p99 latency compared to monolith baseline
- All frontend flows (dashboard + mobile apps) work identically
- RabbitMQ/Kafka message delivery verified with zero dropped events
- Redis cache hit rate > 80% for cached endpoints
- Health checks (liveness + readiness) passing on all pods
- Structured logs flowing to ELK with correct service name and request-id
- Prometheus metrics scraped and Grafana dashboards showing RED metrics

Final Project Acceptance (Week 52)

- All 8 services deployed on Kubernetes with HPA autoscaling
- Zero downtime during rolling updates (PDB + readiness probes verified)
- Distributed tracing working across all service boundaries
- CI/CD pipeline deploying each service independently in < 10 minutes
- Monolith decommissioned, no traffic routed to it
- Runbook documentation for each service (startup, shutdown, troubleshooting)
- Load test results: system handles 2x current peak traffic

Timeline Summary

Scenario	Duration	Recommendation
Full migration (zero feature work)	52 weeks (12 months)	Ideal if team is fully dedicated
With feature work interleaved	15-18 months	More realistic for ongoing product
Phase 0 + 1 only (80/20 rule)	12 weeks (3 months)	80% improvement, lowest risk, start here

Recommendation: Start with Phase 0 + Phase 1 (12 weeks). This delivers structured logging, Redis caching, Docker Compose, graceful shutdown, and decoupled Notification Service — covering 80% of the architectural improvement with the lowest risk. Evaluate whether to continue with Phase 2-7 based on actual scaling needs after the first 12 weeks.

End of Document | Famto Production Microservice Architecture Plan | Confidential | v2.0